Centro Universitario Guadalajara

Lic. En Inteligencia Artificial y Ciencia de Datos

Programación IV

# Tarea 2.1 - PREPARACIÓN - Investigación Pydantic

*Autor:*
Patricia Jaime Pérez

*Profesor:*
Diego Alberto Ayala Vázquez

13 de octubre de 2026

# 1. Marco Teórico: Pydantic y su Rol en FastAPI

## 1.1. ¿Qué es Pydantic?

**Pydantic** es una librería de Python orientada a la **validación de datos y gestión de estructuras**. Su función principal es actuar como un filtro de seguridad que asegura que la información procesada por el sistema cumpla estrictamente con las reglas esperadas. En lugar de verificar manualmente si cada dato enviado por el usuario es válido, Pydantic automatiza este proceso garantizando la integridad de la información en todo momento.

## 1.2. Funciones Principales

- **Validación de Datos:** Comprueba que cada campo contenga el tipo de dato correcto. Si se define un atributo como numérico, Pydantic confirma su validez antes de almacenarlo o procesarlo.
- **Conversión Automática de Formatos (Parsing):** Si un valor ingresa en un formato compatible pero no exacto (por ejemplo, un precio enviado como cadena de texto `"45"`), la librería lo transforma automáticamente al tipo correcto (`45.0`) sin interrumpir la ejecución.
- **Gestión Transparente de Errores:** Ante datos incompletos o con formato incorrecto, genera una respuesta estructurada que detalla con precisión qué campo falló y cuál fue la causa.
- **Definición de Modelos Reutilizables:** Permite diseñar plantillas de datos mediante clases, especificando qué atributos son obligatorios, cuáles opcionales y cuáles tienen valores por defecto.

## 1.3. Funcionamiento de Pydantic dentro de FastAPI

En el desarrollo de la aplicación, Pydantic y FastAPI trabajan de manera integrada en tres etapas clave:

### 1.3.1. 1. Definición del Modelo de Datos (`BaseModel`)

En el módulo del servidor (`main.py`), se hereda de la clase `BaseModel` para construir la estructura formal de las entidades del sistema, como en el caso del objeto `Producto`:

[language=Python] from pydantic import BaseModel

class Producto(BaseModel): id: int nombre: str precio: float

Esta definición establece que todo producto dentro de la aplicación debe contar con un identificador entero (`id`), un nombre en texto (`nombre`) y un precio numérico (`precio`).

### 1.3.2. 2. Intercepción y Validación en Peticiones (`POST`)

Cuando el cliente envía una solicitud HTTP de tipo `POST` para registrar un artículo:

1. El cliente envía los datos en formato JSON desde la interfaz gráfica.
2. **Pydantic intercepta la petición** antes de ejecutar la lógica interna del endpoint y valida que los campos correspondan exactamente con la estructura del modelo.
3. Si los datos son válidos, la petición continúa su flujo normal; de lo contrario, la solicitud se detiene inmediatamente devolviendo una respuesta con código `422 Unprocessable Entity`.

### 1.3.3. 3. Estandarización de Respuestas y Documentación

- **Listados y Consultas (`GET`):** Garantiza que la salida enviada al cliente mantenga siempre una estructura uniforme en formato JSON.
- **Documentación Automática:** FastAPI utiliza las definiciones de Pydantic para interpretar el esquema de datos y generar automáticamente la interfaz interactiva de pruebas en Swagger UI (`http://localhost:8000/docs`).